

Chapter 1

Proximity Service

In this chapter we design a proximity service.

- A proximity service is used to discover nearby places such as restaurants, hotels, theaters, museums, etc., and is a core component that powers features like finding the best restaurants nearby on Yelp or finding k-nearest gas stations on Google Maps.
- Figure 1.1. shows the user interface via which you can search for nearby restaurants on Yelp [1]
- Note the map tiles used in this book are from Stamen Design [2] and data are from OpenStreetMap [3]

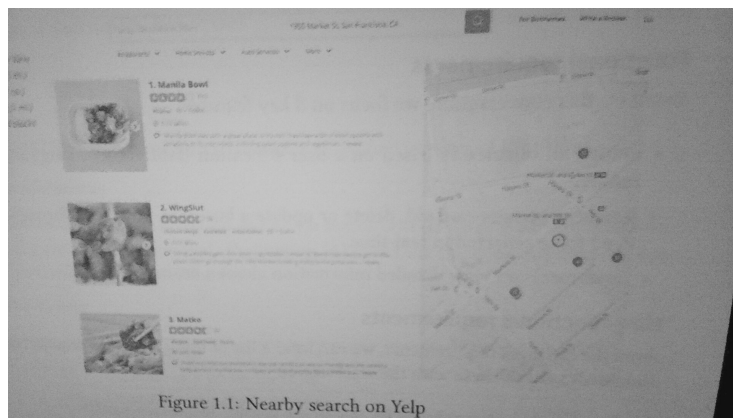


Figure 1.1: Nearby search on Yelp

Nearby search on yelp

Step 1 - Understand the Problem and Establish Design Scope

Yelp supports many features and it is not feasible to design all of them in an interview session, so it's important to narrow down the scope by asking questions

The interactions between the candidate and interviewer could look like this:

- Candidate: Can a user specify the search radius? If there are not enough businesses within the search radius, does the system expand the search?
- Interviewer: That's a great question. Let's assume we only care about businesses within a specified radius. If time allows, we can then discuss how to expand the search if there are not enough businesses within the radius
- Candidate: Can a user change the search radius on the UI?
- Interviewer: Yes, we have the following options: 0.5km (0.31 mile), 1km (0.62 mile), 2km (1.24 mile), 5km (3.1 mile), and 20 km (12.42 mile)
- Candidate: How does business information get added, deleted, or updated? Do we need to reflect these operations in real-time?
- Interviewer: Business owners can add, delete or update a business. Assume we have a business agreement upfront that newly added/updated businesses will be effective the next day.
- Candidate: A user might be moving while using the app/website, so the search results could be slightly different after a while. Do we need to refresh the page to keep the results up to date?
- Interviewer: Let's assume a user's moving speed is slow and we don't need to constantly refresh the page

Functional requirements

Based on this conversation, we focus on 3 key features:

- Return all businesses based on a user's location (latitude and longitude pair) and radius
- Business owners can add, delete, or update a business but this information doesn't need to be reflected in real time
- Customers can view detailed information about a business

Non-functional requirements

From the business requirements, we can infer a list of non-functional requirements. You should also check these with the interviewer.

- Low latency: Users should be able to see nearby businesses quickly

- Data privacy: Location info is sensitive data
 - When we design a location-based service (LBS), we should always take user privacy into consideration
 - We need to comply with data privacy laws like General Data Protection Regulation (GDPR) [4] and California Consumer Privacy Act (CCPA) [5], etc.
 - High availability and scalability requirements: We should ensure our system can handle the spike in traffic during peak hours in densely populated areas

Back-of-the-envelope-estimation

Let's take a look at some back-of-the-envelope-estimation

- Let's take a look at some back-of-the-envelope calculations to determine the potential scale and challenges our solution will need to address
- Assume we have 100 million daily active users and 200 million businesses

Calculate QPS

- Seconds in a day = $24 \times 60 \times 60 = 86400$ seconds. We can round it up to 10^5 for easier calculation
- 10^5 is used in this book for easier calculation and is used throughout this book to represent seconds in a day
- Assume a user makes 5 search queries per day
- Search QPS = $\frac{100 \text{ million} \times 5}{10^5} = 5000$

Table 1.1: Request Parameters

Field	Description	Type
Latitude	Latitude of a given location	decimal
Longitude	Longitude of a given location	decimal
Radius	Optional. Default is 5000 meters	int

```

1  {
2    "total": 18,
3    "businesses": [{business object}]
4  }
```

The business object contains everything needed to render the search result page, but we may still need additional attributes such as pictures, reviews, star rating, etc., to render the business detail page.

Therefore, when a user clicks on the business detail page, a new endpoint call to fetch the detailed information of a business is usually required.

APIs for business

The APIs related to a business object are shown in table below.

Table 1.2: APIs for a business

API	Detail
GET /v1/businesses:id	Return detailed information about a business
POST /v1/businesses	Add a business
PUT /v1/businesses/:id	Update details of a business
DELETE /v1/businesses/:id	DELETE a business

If you are interested in real-world for place/business search, two examples are Google Places API [7] and Yelp business endpoints [8]